Context-Aware Models

Weeks 5-6: From Sequence-to-Sequence to Transformers

PSYC 51.07: Models of Language and Communication

December 28, 2025

Overview 📋

1. 🔄 **Sequence-to-Sequence Models**: The Foundation
2. ⚡ **Attention Mechanisms**: The Key Innovation
3. 🤖 **The Transformer**: Architecture Deep Dive
4. 🎯 **BERT & Variants**: Masked Language Modeling
5. 🧠 **Cognitive Neuroscience**: Prediction in Brains vs. Models
6. ✍️ **Assignment 4**: Building Context-Aware Systems

Goal: Understand how modern models leverage context to understand language

The Context Problem 🤔

Why do we need context-aware models?

The Context Problem 🤔

Why do we need context-aware models?

Example: The word "bank"

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane started to **bank** left" (tilt/turn)

Static Embeddings (Word2Vec):

- One vector per word
- Can't distinguish meanings!
- Context-independent

The Context Problem 🤔

Why do we need context-aware models?

Example: The word "bank"

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane started to **bank** left" (tilt/turn)

Static Embeddings (Word2Vec):

- One vector per word
- Can't distinguish meanings!
- Context-independent

Context-Aware Models:

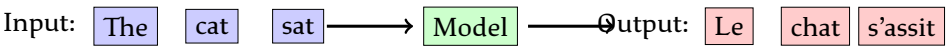
- Different representation per occurrence
- Uses surrounding context
- Handles polysemy naturally

Sequence-to-Sequence Models 🔄

The breakthrough for variable-length input/output problems

Applications:

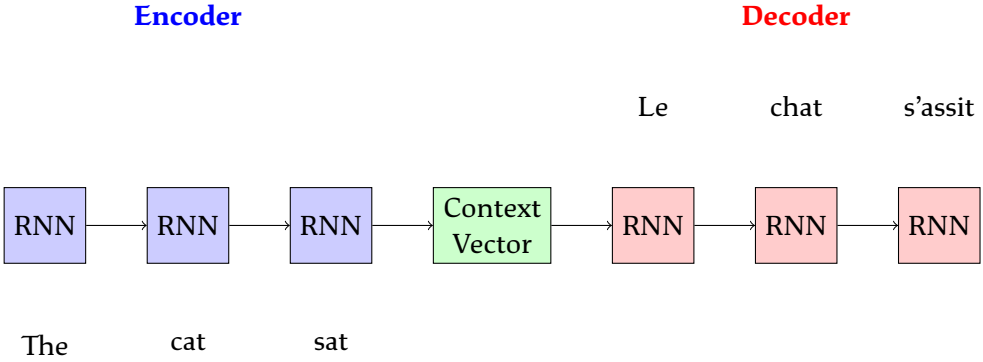
- Machine Translation: English French
- Summarization: Long text Short summary
- Question Answering: Question + Context Answer
- Dialogue Systems: User input System response



Reference: Sutskever et al. (2014) - "Sequence to Sequence Learning with Neural Networks"

Encoder-Decoder Architecture

Two-part architecture: Encode then Decode



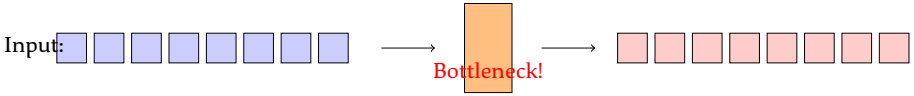
Encoder: **Decoder:**

The Seq2Seq Bottleneck Problem ⚠️

Challenge: All information compressed into single vector!

The Problem

- Long sequences information loss
- Fixed-size context vector is a bottleneck
- Early tokens forgotten by the time we reach the end
- Performance degrades with sequence length

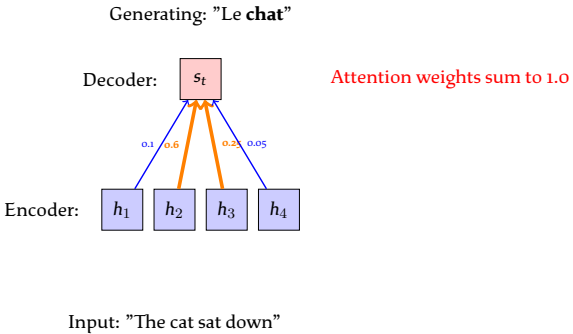


Solution: Attention! ⚡

Attention Mechanism: The Big Idea ⚡

Instead of compressing everything into one vector...

Let the decoder look at all encoder hidden states!



Key Insight: When generating "chat" (cat), pay more attention to "cat" in the input!

Reference: Bahdanau et al. (2015) - "Neural Machine Translation by Jointly Learning to Align and Translate"

How Attention Works: Step by Step 🔍

Computing attention weights:

1. **Score**: How relevant is each encoder state to current decoder state?

$$e_{t,i} = \text{score}(s_t, h_i) = s_t^T W_a h_i$$

2. **Normalize**: Convert scores to probabilities (softmax)

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$$

3. **Context**: Weighted sum of encoder states

$$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i$$

4. **Decode**: Use context vector along with decoder state

$$s_{t+1} = f(s_t, c_t, y_t)$$

Attention Visualization

Example: English French translation with attention weights

Output (French)	Input Words: The	agreement	on	European
L’	0.7	0.1	0.1	0.1
accord	0.1	0.8	0.05	0.05
sur	0.05	0.05	0.8	0.1
l’européen	0.05	0.05	0.1	0.8

Observations:

- Diagonal pattern for similar word order
- Model learns alignment automatically!
- No need for explicit word alignment annotations
- Can handle reordering (syntax differences between languages)

Attention provides interpretability: we can see what the model is "looking at"

The Transformer Revolution 🤖

”Attention Is All You Need”

Before Transformers (2017):

- RNNs/LSTMs with attention
- Sequential processing
- Hard to parallelize
- Limited context window
- Slow training

The Transformer Revolution 🤖

"Attention Is All You Need"

Before Transformers (2017):

- RNNs/LSTMs with attention
- Sequential processing
- Hard to parallelize
- Limited context window
- Slow training

After Transformers:

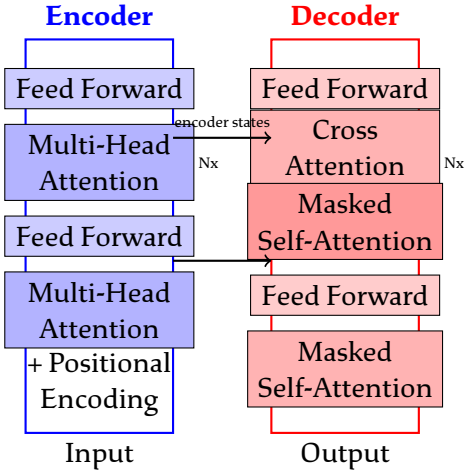
- Pure attention, no recurrence!
- Parallel processing
- Scales to GPUs/TPUs
- Long-range dependencies
- Fast & effective

Key Innovation

Replace sequential RNNs with **self-attention** layers that process all positions in parallel!

Reference: Vaswani et al. (2017) - "Attention Is All You Need"

Transformer Architecture Overview



Key Components:

Self-Attention: The Core Mechanism 🎯

Key idea: Each word attends to all other words in the sequence

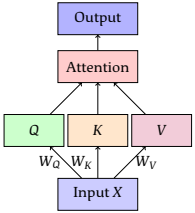
Three learned projections:

- **Query (Q)**: What am I looking for?
- **Key (K)**: What do I contain?
- **Value (V)**: What information do I have?

Computation:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Self-Attention Example

Sentence: "The animal didn't cross the street because it was too tired"

Question: What does "it" refer to?

Self-attention allows the model to:

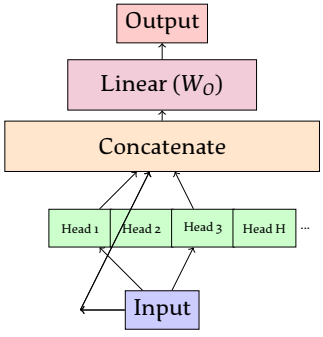
- Resolve pronouns
- Understand long-range dependencies
- Capture syntactic and semantic relationships
- Do this in parallel for all positions!

Multi-Head Attention 🎭

Why use multiple attention heads?

Intuition:

- Different heads learn different relationships
- Head 1: Syntactic relationships
- Head 2: Semantic relationships
- Head 3: Positional patterns
- Etc.



Formula:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(h_1, ..., h_H)W_O$$
$$h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Typical: 8-16 heads in practice

BERT-base: 12 heads, BERT-large: 16 heads, GPT-3: 96 heads!

Three Types of Attention 🔍

1. **Self-Attention (Encoder)**
   - Each position attends to all positions in same sequence
   - Bidirectional: can see past and future
   - Used in: BERT, encoder-only models
2. **Masked Self-Attention (Decoder)**
   - Each position attends only to previous positions
   - Prevents ”looking into the future”
   - Used in: GPT, decoder-only models
3. **Cross-Attention (Encoder-Decoder)**
   - Decoder attends to encoder outputs
   - Queries from decoder, Keys/Values from encoder
   - Used in: T5, BART, machine translation

Type	Q from	K, V from
Self-Attention	Sequence	Same sequence
Masked Self-Attention	Sequence	Same sequence (past only)
Cross-Attention	Decoder	Encoder

Positional Encoding 📍

Problem: Self-attention is permutation-invariant!

Without position information:

- "The cat sat" = "sat cat the" = "cat the sat"
- Order matters in language!

Solution: Add positional encodings to input embeddings

Original Transformer (Sinusoidal):

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

Properties:

- Deterministic
- Unique for each position
- Smooth changes
- Generalizes to unseen lengths

Modern Approach: RoPE

Rotary Position Embedding (Su et al. 2021)

- Rotates Q and K in complex space
- Relative position encoding
- Better extrapolation
- Used in: LLaMA, GPT-NeoX

Learned Positional Embeddings:

- Treat positions as vocabulary
- Learn embeddings during training
- Used in: BERT, GPT

References: Vaswani et al. (2017), Su et al. (2021) - "RoFormer: Enhanced Transformer with Rotary Position Embedding"

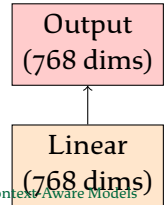
Feed-Forward Networks 🔄

After attention, apply position-wise feed-forward network

Architecture:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- Two linear transformations with ReLU activation
- Applied to each position independently
- Same network for all positions
- Typical: Expand 4x then project back



Layer Normalization & Residual Connections 🔗

Critical for training deep transformers!

Residual Connections:

$$\text{Output} = \text{Layer}(x) + x$$

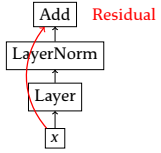
- Allows gradients to flow directly
- Prevents vanishing gradients
- Enables very deep networks
- Identity mapping as fallback

Analogy: Like highway roads for information flow!

Layer Normalization:

$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sigma} + \beta$$

- Normalize across features
- Stabilizes training
- Faster convergence
- Independent of batch size



Standard Pattern:

$$x \leftarrow \text{LayerNorm}(x + \text{MultiHeadAttention}(x))$$
$$x \leftarrow \text{LayerNorm}(x + \text{FFN}(x))$$

FlashAttention: Making Transformers Faster ⚡

Problem: Standard attention is slow and memory-hungry!

Standard Attention Complexity

- Time: $O(n^2)$ where n is sequence length
- Memory: $O(n^2)$ to store attention matrix
- Bottleneck: Reading/writing to GPU memory (HBM)

FlashAttention Innovation:

- Tile-based computation
- Uses fast SRAM instead of slow HBM
- Fused operations
- Recomputation in backward pass
- 2-4x faster!
- Enables longer sequences

Impact:

- GPT-3: 512 [] 2048 context
- BERT: Faster training
- Long-document understanding
- Used in: LLaMA, GPT-4

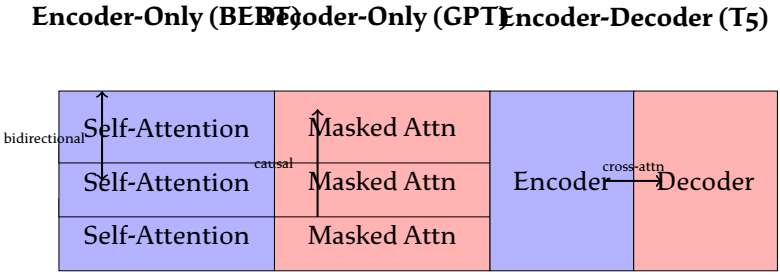
Method	Speed	Memory
Standard	1x	1x
FlashAttn	**3x**	**0.5x**

Reference: Dao et al. (2022) - "FlashAttention: Fast and Memory-Efficient Exact Attention"

Three Transformer Architectures

Architecture	How it works	Examples & Uses
Encoder-only *Best for:* Classification, NER, QA	Bidirectional self-attention. Can see entire sequence.	BERT, RoBERTa, ALBERT
Decoder-only *Best for:* Generation, completion	Masked self-attention. Can only see past. Autoregressive.	GPT, GPT-2, GPT-3, LLaMA
Encoder-Decoder Decoder: masked + cross-attention	Encoder: bidirectional	T5, BART, mT5

Visual Comparison: Encoder vs Decoder vs Both



Choosing the Right Architecture:

- Need to understand full context? → **Encoder** (BERT)
- Need to generate text? → **Decoder** (GPT)
- Need both (translate, summarize)? → **Encoder-Decoder** (T5)

Using Transformers in Practice 💻

HuggingFace makes it easy!

```
from transformers import AutoModel, AutoTokenizer

# Load pre-trained model
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Tokenize input
text = "The animal didn't cross the street because it was too
    tired"
inputs = tokenizer(text, return_tensors="pt")

# Get contextualized embeddings
outputs = model(**inputs)
hidden_states = outputs.last_hidden_state  # Shape: [batch,
    seq_len, 768]
```

BERT: Bidirectional Encoder Representations 🎯

BERT = Encoder-only Transformer

Key Innovation: Masked Language Modeling (MLM)

Traditional Language Models:

- Left-to-right (GPT)
- Or right-to-left
- Unidirectional context
- Can't see full picture

Example:

- "The cat sat on the ___"
- Only sees left context

BERT: Bidirectional Encoder Representations 🎯

BERT = Encoder-only Transformer

Key Innovation: Masked Language Modeling (MLM)

Traditional Language Models:

- Left-to-right (GPT)
- Or right-to-left
- Unidirectional context
- Can't see full picture

Example:

- "The cat sat on the ___"
- Only sees left context

BERT (MLM):

- Mask random tokens
- Predict from bidirectional context
- Sees both left & right
- Deeper understanding

Example:

- "The cat [MASK] on the mat"
- Sees: "The cat" AND "on the mat"
- Predicts: "sat"

Reference: Devlin et al. (2019) - "BERT: Pre-training of Deep Bidirectional Transformers"

Masked Language Modeling (MLM) 🎭

BERT's Pre-training Objective

Training Procedure:

1. Take a sentence
2. Randomly mask 15% of tokens
3. Of the masked tokens:
   - 80%: Replace with [MASK]
   - 10%: Replace with random word
   - 10%: Keep unchanged
4. Predict the original tokens

Example

Original: "My dog is hairy"
Masked: "My dog is [MASK]"
Labels: [-, -, -, hairy]
Prediction: Model predicts "hairy" using bidirectional context

Why the 80/10/10 split?

- Prevents overfitting to [MASK] token
- Forces model to maintain representations for all tokens

BERT Architecture Variants

Model	Layers	Hidden Size	Heads	Params
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M
RoBERTa-Base	12	768	12	125M
RoBERTa-Large	24	1024	16	355M
ALBERT-Base	12	768	12	12M
DistilBERT	6	768	12	66M

Architecture Details (BERT-Base):

- 12 transformer encoder layers
- 768-dimensional hidden states
- 12 attention heads per layer
- 3072-dimensional feed-forward intermediate size
- Maximum sequence length: 512 tokens
- Vocabulary size: 30,000 WordPiece tokens

BERT Variants & Improvements

1. **RoBERTa (Liu et al. 2019)**
   - Remove Next Sentence Prediction (NSP)
   - Train longer with bigger batches
   - Dynamic masking
   - Better performance than BERT
2. **ALBERT (Lan et al. 2019)**
   - Parameter sharing across layers
   - Factorized embedding parameterization
   - 12M params vs BERT's 110M
   - Similar performance, much smaller
3. **DistilBERT (Sanh et al. 2019)**
   - Knowledge distillation from BERT
   - 6 layers instead of 12
   - 40% smaller, 60% faster
   - Retains 97% of performance

References: Liu et al. (2019) - RoBERTa, Sanh et al. (2019) - DistilBERT

Fine-tuning BERT 🎓

Two-stage process: Pre-train then Fine-tune

```
from transformers import BertForSequenceClassification, Trainer,
    TrainingArguments

# Load pre-trained BERT with classification head
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2  # Binary classification
)

# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
```

BERT Applications 🚀

BERT excels at understanding tasks:

Classification Tasks:

- Sentiment Analysis
- Topic Classification
- Spam Detection
- Intent Recognition

Token-Level Tasks:

- Named Entity Recognition (NER)
- Part-of-Speech Tagging
- Word Sense Disambiguation

Span-Level Tasks:

- Question Answering
- Extractive Summarization
- Information Extraction

Sentence-Pair Tasks:

- Semantic Similarity
- Natural Language Inference
- Paraphrase Detection

Industry Impact

BERT powers:

- Google Search (understanding queries)
- Customer service chatbots
- Content moderation
- Document understanding

Contextual Embeddings in Action

Remember "bank"? Let's see BERT handle it!

```
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

# Two different contexts for "bank"
sent1 = "I deposited money at the bank"
sent2 = "We sat by the river bank"

# Get embeddings
def get_embedding(sentence, target_word):
    inputs = tokenizer(sentence, return_tensors='pt')
    outputs = model(**inputs)
    # Find position of target word
    tokens = tokenizer.tokenize(sentence)
```

Cognitive Neuroscience Perspective 🧠

How do brains and models process language?

Predictive Processing in the Brain:

- Brain constantly predicts upcoming input
- N400: Neural response to unexpected words
- P600: Syntactic anomaly detection
- Context shapes predictions
- Prediction errors drive learning

Key Brain Regions:

- **Left IFG**: Syntax processing
- **Left STG/MTG**: Semantic processing
- **ATL**: Conceptual knowledge

Predictive Processing in Models:

- BERT: Predict masked words
- GPT: Predict next word
- Both use context to predict
- Surprise = high loss
- Gradient descent = learning

Similarities:

- Both hierarchical
- Both context-sensitive
- Both predictive
- Both learn from errors

References: Kuperberg & Jaeger (2016), Willems et al. (2016), Hagoort & Indefrey (2014)

Prediction in Brains vs. Language Models 🧠🤖

Parallels between neural and artificial systems

Human Brain	Transformer Models
N400 component (surprise at unexpected word) (low probability prediction)	High cross-entropy loss
Hierarchical processing (sounds → words → sentences) (tokens → phrases → meaning)	Layer-by-layer representations
Context integration (prior discourse, world knowl-edge) (attend to relevant context)	Self-attention mechanism

Neural Encoding with Language Models

Can we predict brain activity from language models?

Approach:

1. Show participants text while recording brain activity (fMRI/EEG)
2. Extract representations from language model (e.g., BERT layer 6)
3. Train regression model: Model embeddings Brain activity
4. Test: Can we predict brain responses to new text?

Findings:

- **Better models Better brain prediction**
- BERT outperforms Word2Vec at predicting brain activity
- Different layers correlate with different brain regions
- Middle layers best for semantic areas
- Suggests shared representations?

But...

Correlation Causation! Models might capture statistical regularities that brains use, without using the same mechanisms.

Reference: Willems et al. (2016) - "Prediction during natural language comprehension"

Discussion: What Does the Model "Understand"? 🤔

Does BERT understand language?

Evidence FOR understanding:

- Captures syntax and semantics
- Resolves ambiguity
- Handles long-range dependencies
- Generalizes to new examples
- Predicts brain activity
- Solves complex tasks

"If it acts like it understands, maybe it does?"

Evidence AGAINST understanding:

- No grounding in physical world
- No sensory experience
- No social context
- Brittle to adversarial examples
- No common sense reasoning
- Only pattern matching?

"Understanding requires more than statistical patterns"

Key Questions

- What is the difference between understanding and correlation?
- Can meaning exist without grounding?
- Is human understanding fundamentally different?

Class Discussion: What do YOU think?

Limitations of Current Models ⚠️

Despite impressive performance, transformers have limitations:

1. **Quadratic Complexity**
   - Self-attention scales as $O(n^2)$
   - Limited context windows (512-4096 tokens)
   - Cannot process very long documents efficiently
2. **No True Understanding**
   - Pattern matching vs. comprehension
   - Lack of common sense
   - No world model
3. **Data Efficiency**
   - Requires massive training data
   - Humans learn language with much less data
   - Not biologically plausible
4. **Biases and Fairness**
   - Inherits biases from training data
   - Can amplify stereotypes
   - Ethical concerns

Interactive Learning Resources 🎓

Great tutorials for deeper understanding:

1. **The Illustrated Transformer**
   - Blog post by Jay Alammar
   - Visual step-by-step walkthrough
   - https://jalammar.github.io/illustrated-transformer/
2. **Animated Transformer Tutorial**
   - Interactive visualizations
   - Watch attention in action
   - https://transformer-circuits.pub/
3. **HuggingFace Course**
   - Chapter 1.4: How do Transformers work?
   - Hands-on coding tutorials
   - https://huggingface.co/course/chapter1/4
4. **Attention Mechanism Visualization**
   - BertViz: Visualize BERT attention
   - See which words attend to which
   - https://github.com/jessevig/bertviz

Comparing Architectures: Code Examples 💻

Different architectures for different tasks

```
from transformers import AutoModelForSequenceClassification, \
                         AutoModelForCausalLM, \
                         AutoModelForSeq2SeqLM

# 1. ENCODER-ONLY (BERT): Classification
encoder_model = AutoModelForSequenceClassification.from_pretrained
    (
     "bert-base-uncased", num_labels=2
)
# Use for: Sentiment analysis, NER, classification

# 2. DECODER-ONLY (GPT): Text generation
decoder_model = AutoModelForCausalLM.from_pretrained("gpt2")
# Use for: Text completion, creative writing, few-shot learning

# 3. ENCODER-DECODER (T5): Seq2Seq tasks
```

Assignment 4: Context-Aware Models 📝

Hands-on experience with transformers!

Tasks:

1. **Implement Attention Mechanism**
   - Build scaled dot-product attention from scratch
   - Visualize attention weights
2. **Fine-tune BERT**
   - Load pre-trained BERT
   - Fine-tune on sentiment analysis
   - Compare to baseline models
3. **Analyze Contextual Embeddings**
   - Extract embeddings for polysemous words
   - Visualize how context changes representations
   - Compare BERT vs Word2Vec
4. **Explore Different Architectures**
   - Compare BERT (encoder) vs GPT (decoder)
   - Test on different tasks
   - Analyze strengths/weaknesses
5. **Research Component**
   - Read one paper from references
   - Write brief summary & critical analysis

Due: Check course website for deadline

Practical Tips for Working with Transformers 💡

1. **Start with Pre-trained Models**
   - Don't train from scratch (too expensive!)
   - Use HuggingFace Model Hub
   - Choose appropriate model size
2. **Fine-tuning Best Practices**
   - Use small learning rate (1e-5 to 5e-5)
   - Add warmup steps
   - Monitor for overfitting
   - Freeze early layers if data is limited
3. **Computational Efficiency**
   - Use mixed precision training (FP16)
   - Gradient accumulation for larger batch sizes
   - Consider DistilBERT for faster inference
   - Use FlashAttention when available
4. **Evaluation**
   - Use task-specific metrics
   - Test on out-of-distribution data
   - Check for biases
   - Visualize attention for interpretability

Resources & Further Reading

Key Papers:

- Sutskever et al. (2014) - Sequence to Sequence Learning
- Bahdanau et al. (2015) - Neural Machine Translation by Jointly Learning to Align
- Vaswani et al. (2017) - Attention Is All You Need
- Devlin et al. (2019) - BERT: Pre-training of Deep Bidirectional Transformers
- Liu et al. (2019) - RoBERTa: A Robustly Optimized BERT Pretraining Approach
- Sanh et al. (2019) - DistilBERT: A distilled version of BERT
- Su et al. (2021) - RoFormer: Enhanced Transformer with Rotary Position Embedding
- Dao et al. (2022) - FlashAttention: Fast and Memory-Efficient Exact Attention

Cognitive Neuroscience:

- Hagoort & Indefrey (2014) - The neurobiology of language beyond single words
- Kuperberg & Jaeger (2016) - What do we mean by prediction in language comprehension?
- Willems et al. (2016) - Prediction during natural language comprehension

Tutorials:

- HuggingFace Course: Chapters 1.4, 1.5, 7.3
- The Illustrated Transformer: `https://jalammar.github.io/illustrated-transformer/`
- Animated Transformer: `https://transformer-circuits.pub/`

Summary 🎯

What we learned:

1. **Evolution of Context**
   - Seq2Seq Attention Transformers
   - From bottleneck to full parallelization
2. **Attention Mechanisms**
   - Self-attention, masked attention, cross-attention
   - Query, Key, Value framework
   - Multi-head attention for diverse relationships
3. **Transformer Architecture**
   - Pure attention-based architecture
   - Encoder-only (BERT), Decoder-only (GPT), Both (T5)
   - Positional encoding, layer norm, residual connections
4. **BERT & Masked Language Modeling**
   - Bidirectional context from MLM objective
   - Pre-train then fine-tune paradigm
   - Variants: RoBERTa, ALBERT, DistilBERT
5. **Brain-Model Parallels**
   - Both use prediction
   - Both hierarchical and context-sensitive
   - Deep questions about understanding

Looking Forward 🔮

Where do we go from here?

Next Topics:

- Scaling laws: Bigger models, better performance?
- Efficient transformers: Linear attention, sparse attention
- Multimodal transformers: Vision + Language
- In-context learning: GPT-3 and beyond
- Alignment: Making models helpful and safe

Open Questions:

- How can we make transformers more efficient?
- Can we combine symbolic reasoning with neural networks?
- How do we ensure fairness and reduce bias?
- What is the role of grounding in understanding?
- Are we building towards AGI, or is something missing?

The transformer revolution continues! 🚀

Questions? 🙋

Discussion Time

Topics to discuss:

- Attention mechanisms
- Transformer architectures
- BERT vs GPT: When to use which?
- Do models really ”understand”?
- Assignment 4 questions

Thank you! 🙏